Projet DevOps : Bibliothèque Numérique Microservices

Master 1 Intelligence Artificielle

Groupe	Groupe 2
Membres	AHOGA Josias BAH Mamoudou BAMBA Yannick DIOP Seynabou

Technologies	FastAPI · PostgreSQL · Docker · Jenkins
Architecture	Microservices · REST API · CI/CD
Livrables	Code source GitHub + Rapport PDF + README

Table des matières

1. Présentation du projet
    1.1 Contexte et problématique
    1.2 Objectifs pédagogiques
2. Architecture du système
    2.1 Vue d'ensemble
    2.2 Choix technologiques
3. Description des microservices
    3.1 Service Livres (port 8001)
    3.2 Service Utilisateurs (port 8002)
    3.3 Service Emprunts (port 8003)
    3.4 Frontend (port 80)
4. Conteneurisation Docker
    4.1 Dockerfiles
    4.2 Docker Compose
5. Pipeline CI/CD Jenkins
    5.1 Structure du pipeline
    5.2 Étapes détaillées
6. Guide d'installation
8. Conclusion

1. Présentation du projet

1.1 Contexte et problématique

Le Dakar Institute of Technology (DIT) souhaitait moderniser la gestion de sa bibliothèque académique. La gestion manuelle des livres et des emprunts engendrait plusieurs problèmes opérationnels : difficultés de suivi des livres disponibles, absence de statistiques fiables sur les emprunts, gestion inefficace des retards, et manque d'accès numérique pour les étudiants et enseignants.

Ce projet répond à ces besoins en développant une plateforme web moderne basée sur une architecture microservices, entièrement conteneurisée et déployée via un pipeline CI/CD automatisé.

1.2 Objectifs pédagogiques

Objectif	Réalisation
Concevoir une architecture microservices	Découpage en 3 services indépendants
Développer une API backend	FastAPI (Python) avec PostgreSQL
Concevoir une interface frontend	HTML / CSS / JavaScript pur
Utiliser Docker et Docker Compose	Conteneurisation complète
Mettre en place un pipeline CI/CD	Jenkins avec Jenkinsfile
Gérer un projet avec Git et GitHub	Dépôt organisé avec README
Automatiser le déploiement	Pipeline end-to-end

2. Architecture du système

2.1 Vue d'ensemble

L'application est structurée selon le pattern microservices : chaque fonctionnalité métier est encapsulée dans un service autonome. Les services communiquent entre eux via des API REST. L'ensemble est orchestré par Docker Compose et déployé via un pipeline Jenkins.

Couche	Composant	Port	Rôle
Présentation	Frontend (nginx)	80	Interface web HTML/CSS/JS
Métier	Service Livres	8001	CRUD livres, recherche
Métier	Service Utilisateurs	8002	Membres, profils, types
Métier	Service Emprunts	8003	Prêts, retours, retards
Données	PostgreSQL 16	5432	Base de données relationnelle
CI/CD	Jenkins	8080	Pipeline automatisé

2.2 Choix technologiques

Les technologies ont été choisies pour leur simplicité de prise en main et leur pertinence dans un contexte DevOps réel :

Technologie	Version	Justification
FastAPI	0.115	Framework Python moderne, documentation auto, performant
PostgreSQL	16	SGBD relationnel robuste, conforme aux standards SQL
Docker	latest	Conteneurisation standard de l'industrie
Docker Compose	v3.8	Orchestration multi-conteneurs locale
Jenkins	LTS	Serveur CI/CD open source, très répandu
Nginx	alpine	Serveur web léger pour le frontend statique

3. Description des microservices

3.1 Service Livres (port 8001)

Le service Livres gère l'intégralité du catalogue de la bibliothèque. Chaque livre est décrit par un titre, un auteur, un ISBN unique, une année de publication, un genre et un statut de disponibilité. Le statut est automatiquement mis à jour lors des emprunts et retours par le service Emprunts.

Structure de la table livres :

id	Clé primaire auto-incrémentée
titre	Titre du livre (obligatoire)
auteur	Nom de l'auteur (obligatoire)
isbn	Identifiant ISBN unique (obligatoire)
annee	Année de publication (optionnel)
genre	Catégorie du livre (optionnel)
disponible	True = disponible, False = emprunté

Endpoints REST :

Méthode	Endpoint	Description
GET	/livres	Lister tous les livres
GET	/livres/recherche	Rechercher par titre, auteur ou ISBN
GET	/livres/{id}	Obtenir un livre par son identifiant
POST	/livres	Ajouter un nouveau livre
PUT	/livres/{id}	Modifier un livre existant
DELETE	/livres/{id}	Supprimer un livre
GET	/health	Vérification de l'état du service

3.2 Service Utilisateurs (port 8002)

Le service Utilisateurs gère les membres inscrits à la bibliothèque. Trois types d'utilisateurs sont supportés : étudiant, professeur et personnel administratif. Chaque membre dispose d'un matricule unique qui l'identifie dans le système.

Endpoints REST :

Méthode	Endpoint	Description
GET	/utilisateurs	Lister tous les membres
GET	/utilisateurs/type/{type}	Filtrer par type (etudiant, professeur…)
GET	/utilisateurs/{id}	Consulter le profil d'un membre
POST	/utilisateurs	Inscrire un nouveau membre
PUT	/utilisateurs/{id}	Modifier un profil
DELETE	/utilisateurs/{id}	Supprimer un membre
GET	/health	Vérification de l'état du service

3.3 Service Emprunts (port 8003)

Le service Emprunts est le plus complexe : il orchestre les deux autres services. Lors d'un emprunt, il vérifie que l'utilisateur existe (appel au service Utilisateurs) et que le livre est disponible (appel au service Livres), puis crée l'enregistrement et marque automatiquement le livre comme indisponible. La détection des retards est automatique : lors du retour, la date réelle est comparée à la date prévue.

Logique métier d'un emprunt :

- 1. Vérification de l'existence de l'utilisateur via service-utilisateurs
- 2. Vérification de la disponibilité du livre via service-livres
- 3. Création de l'enregistrement d'emprunt (date + 14 jours par défaut)
- 4. Mise à jour du statut du livre : disponible = False
- 5. Retour de la confirmation à l'appelant

Endpoints REST :

Méthode	Endpoint	Description
POST	/emprunts	Enregistrer un nouvel emprunt
PUT	/emprunts/{id}/retour	Enregistrer le retour d'un livre
GET	/emprunts	Historique complet des emprunts
GET	/emprunts/en-cours	Emprunts non encore retournés
GET	/emprunts/retards	Emprunts en retard (date dépassée)
GET	/emprunts/utilisateur/{id}	Historique d'un membre spécifique
GET	/health	Vérification de l'état du service

3.4 Frontend (port 80)

L'interface frontend est développée en HTML, CSS et JavaScript vanille, servie par un conteneur Nginx. Elle communique directement avec les trois microservices via l'API fetch() du navigateur. Elle propose quatre sections accessibles depuis une barre de navigation latérale :

Livres	Catalogue complet, recherche en temps réel, ajout et suppression de livres
Utilisateurs	Liste des membres, inscription, gestion par type
Emprunts	Enregistrement des emprunts et retours, historique
Retards	Tableau de bord des livres non rendus avec calcul des jours de retard

4. Conteneurisation Docker

4.1 Dockerfiles

Chaque composant possède son propre Dockerfile. Les services FastAPI utilisent une image Python 3.12 allégée (slim), le frontend utilise Nginx Alpine pour sa légèreté. La stratégie de construction en deux étapes (copie requirements puis code) optimise le cache Docker et accélère les builds suivants.

Exemple : Dockerfile service-livres

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8001
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]
```

4.2 Docker Compose

Le fichier docker-compose.yml orchestre les 5 conteneurs (3 microservices + frontend + PostgreSQL). Les points clés de la configuration sont :

Healthcheck PostgreSQL : Le service postgres est déclaré healthy uniquement quand pg_isready retourne 0, garantissant que les services démarrent dans le bon ordre.

Dépendances ordonnées : Les services backend attendent que PostgreSQL soit healthy. Le service-emprunts attend en plus les deux autres services.

Réseau interne : Tous les services partagent le réseau bibliotheque-network. Les noms de conteneurs servent de noms de domaine internes (ex : http://service-livres:8001).

Volume persistant : Le volume postgres_data persiste les données même si les conteneurs sont arrêtés ou reconstruits.

Variables d'environnement : Les URLs des services et les credentials PostgreSQL sont injectés via des variables d'environnement, sans être codés en dur.

Commande de lancement :

```
docker compose up --build
docker compose down
```

5. Pipeline CI/CD Jenkins

5.1 Structure du pipeline

Le pipeline est défini dans un fichier Jenkinsfile à la racine du projet. Jenkins le détecte automatiquement lors de la configuration du job. À chaque push sur GitHub, le pipeline s'exécute de bout en bout.

Étape	Nom	Action
1	Récupération du code	checkout scm depuis GitHub
2	Vérification du projet	Contrôle la présence des Dockerfiles et du compose
3	Build des images Docker	docker compose build --no-cache
4	Arrêt ancienne version	docker compose down --remove-orphans
5	Déploiement	docker compose up -d
6	Vérification santé	curl sur les endpoints /health de chaque service

5.2 Gestion des succès et échecs

Le bloc post du Jenkinsfile définit les actions à exécuter selon le résultat du pipeline. En cas de succès, un message de confirmation est affiché avec l'URL d'accès. En cas d'échec, les conteneurs sont arrêtés proprement pour libérer les ressources et éviter de laisser une version instable en production.

Le bloc **always** s'exécute dans tous les cas et produit un résumé dans les logs Jenkins, permettant un audit rapide de chaque déploiement.

6. Guide d'installation

Prérequis

- Docker Desktop (inclut Docker Compose)
- Git
- Un navigateur web moderne

Étapes de déploiement

Étape 1 - Cloner le dépôt GitHub

```
git clone https://github.com/Sihiney/bibliotheque-dit.git
```

Étape 2 - Se placer dans le dossier du projet

```
cd bibliotheque-dit
```

Étape 3 - Lancer l'application complète

```
docker compose up --build
```

Étape 4 - Accéder à l'interface web

```
http://localhost
```

Étape 5 - Accéder à la documentation API

```
http://localhost:8001/docs (Livres) http://localhost:8002/docs (Utilisateurs)
http://localhost:8003/docs (Emprunts)
```

Structure du dépôt GitHub

Dossier / Fichier	Contenu
service-livres/	main.py · models.py · schemas.py · database.py · Dockerfile
service-utilisateurs/	Même structure que service-livres
service-emprunts/	Même structure + httpx pour appels inter-services
frontend/	index.html (interface complète) · Dockerfile (nginx)
docker-compose.yml	Orchestration des 5 conteneurs
Jenkinsfile	Définition du pipeline CI/CD en 6 étapes
README.md	Documentation d'installation et d'utilisation

7. Captures d'ecran de l'application

Les captures suivantes illustrent l'application deployee et fonctionnelle sur un environnement local. Chaque page de l'interface est presentee avec des donnees reelles.

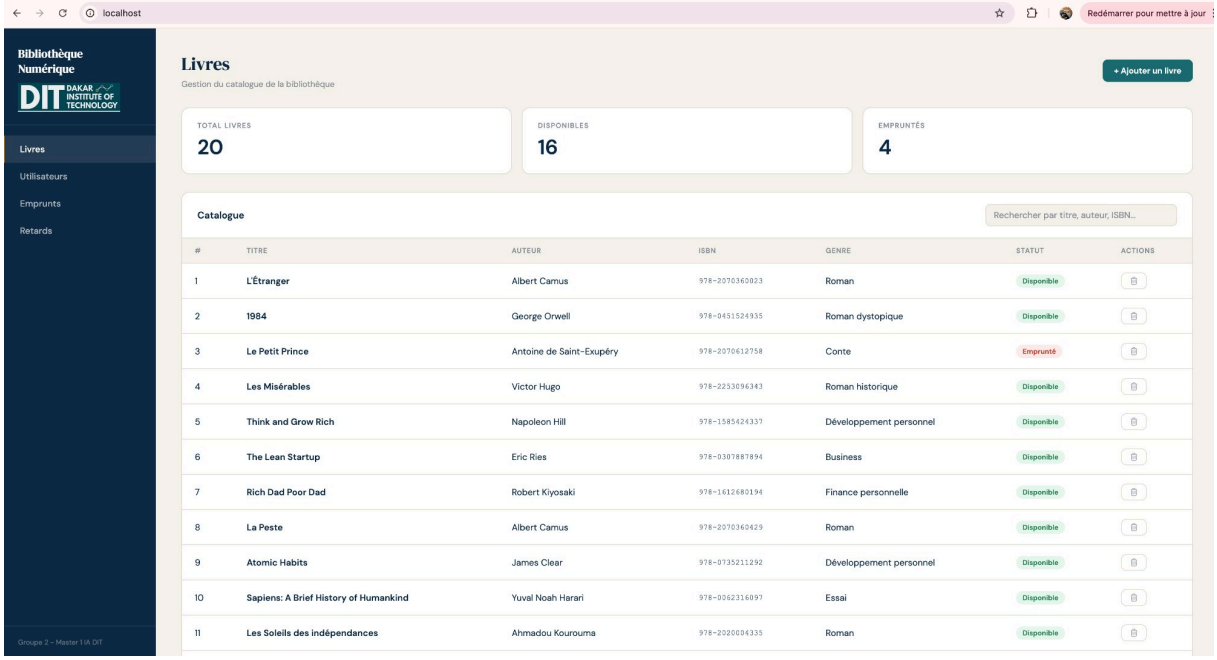

Figure 1 - Page Livres : catalogue complet avec statistiques (20 livres, 16 disponibles, 4 empruntés)

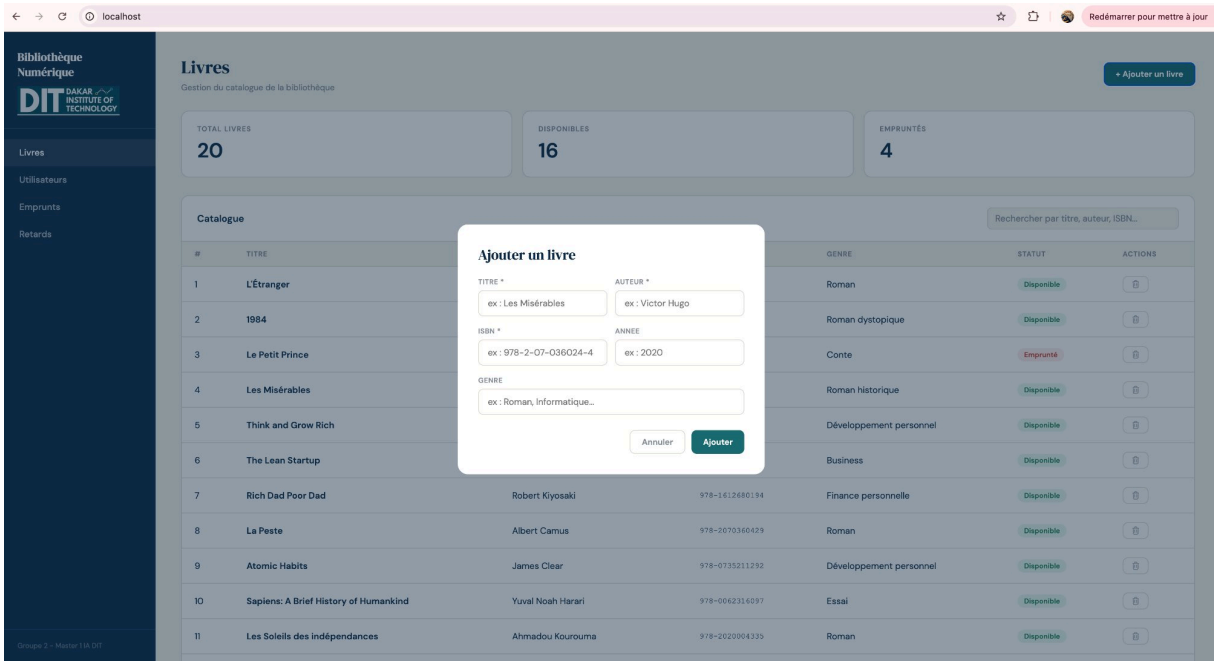

Figure 2 - Page Livres : formulaire d'ajout d'un nouveau livre

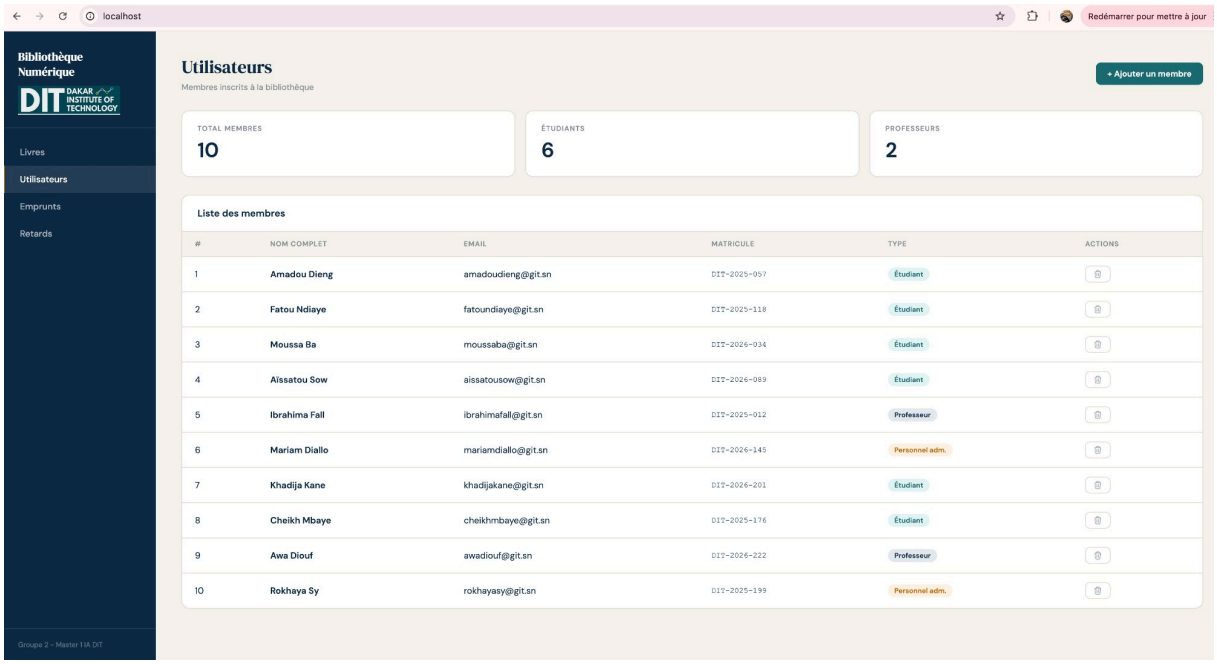

Figure 3 - Page Utilisateurs : liste des 10 membres avec types et matricules

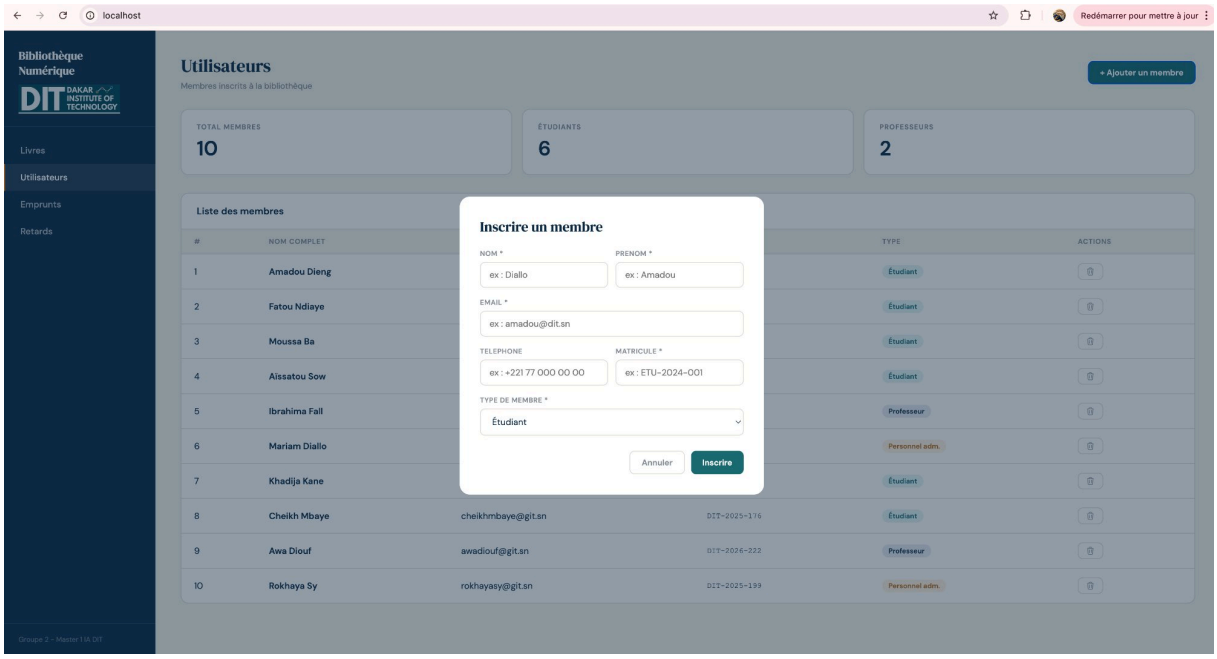

Figure 4 - Page Utilisateurs : formulaire d'inscription d'un nouveau membre

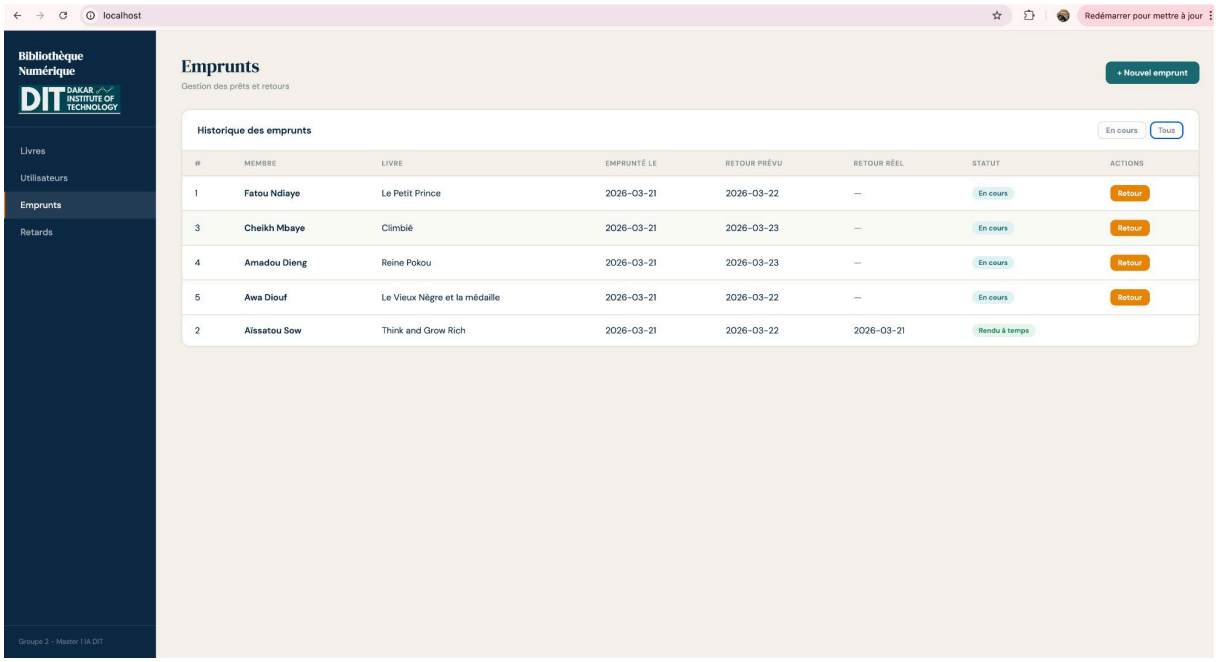

Figure 5 - Page Emprunts : historique complet des emprunts (en cours et passés)

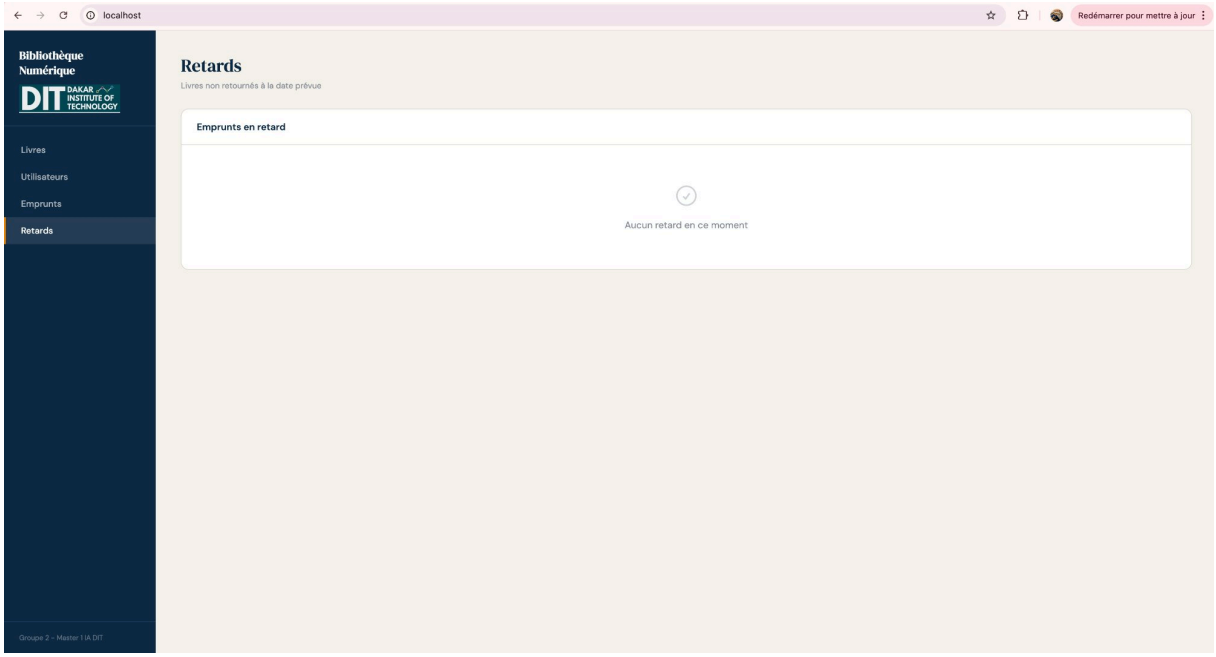

Figure 6 - Page Retards : aucun retard enregistré pour le moment

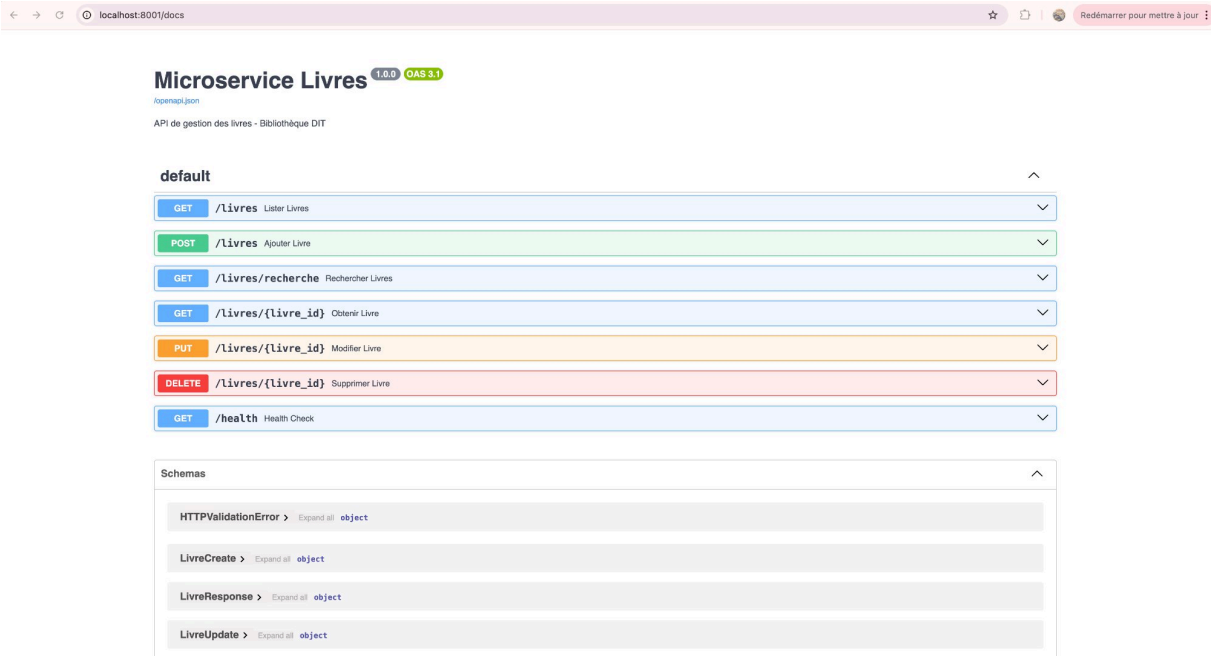

Figure 7 - Documentation API interactive générée automatiquement par FastAPI (Microservice Livres)

8. Conclusion

Ce projet a permis à notre groupe de mettre en pratique l'ensemble des compétences DevOps visées par le programme du Master 1 Intelligence Artificielle du DIT. L'application développée répond intégralement aux exigences fonctionnelles et techniques du cahier des charges.

L'architecture microservices adoptée offre plusieurs avantages concrets : indépendance des services, scalabilité individuelle, et isolation des pannes. La conteneurisation Docker garantit la portabilité et la reproductibilité des déploiements sur tout environnement.

Le pipeline Jenkins automatise intégralement le cycle de vie du déploiement : du push GitHub jusqu'à la vérification de santé des services en production. Cette approche CI/CD est aujourd'hui la norme dans l'industrie du développement logiciel.

Critère	Statut
Architecture microservices	Réalisé - 3 services indépendants
API REST complète	Réalisé - CRUD + endpoints métier
Interface frontend	Réalisé - 4 pages fonctionnelles
Dockerfiles	Réalisé - 1 Dockerfile par composant
Docker Compose	Réalisé - Orchestration complète
Pipeline Jenkins	Réalisé - 6 étapes automatisées
README	Réalisé - Installation et utilisation
Rapport PDF	Réalisé - Document présent